

# TreasureMaze: Progetto d'esame Introduzione all'Intelligenza artificiale

Pietro Bonelli

A.A. 2025-26

## Indice

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                         | <b>2</b>  |
| 1.1      | Descrizione dei Vincoli . . . . .                           | 2         |
| 1.2      | Descrizione del Dominio . . . . .                           | 3         |
| <b>2</b> | <b>Rappresentazione degli Stati e Spazio degli Stati</b>    | <b>3</b>  |
| 2.1      | Metodi della classe <code>Problem</code> . . . . .          | 4         |
| 2.2      | Stato iniziale e Goal test . . . . .                        | 4         |
| <b>3</b> | <b>Algoritmi di Ricerca</b>                                 | <b>4</b>  |
| 3.1      | Ricerca Non Informata: Breadth-First Search (BFS) . . . . . | 4         |
| 3.2      | Ricerca Informata: Algoritmo $A^*$ . . . . .                | 4         |
| 3.3      | Euristiche . . . . .                                        | 5         |
| <b>4</b> | <b>Computer Vision</b>                                      | <b>5</b>  |
| 4.1      | Pre-elaborazione e Binarizzazione . . . . .                 | 5         |
| 4.2      | Analisi morfologica e Perimetrale . . . . .                 | 6         |
| 4.3      | Segmentazione delle Celle . . . . .                         | 6         |
| 4.4      | Estrazione della Matrice Simbolica . . . . .                | 6         |
| <b>5</b> | <b>Classificazione</b>                                      | <b>7</b>  |
| 5.1      | Dataset e Pre-Processing . . . . .                          | 7         |
| 5.2      | Architettura del Modello . . . . .                          | 7         |
| 5.3      | Addestramento . . . . .                                     | 8         |
| <b>6</b> | <b>Risultati e Statistiche</b>                              | <b>8</b>  |
| 6.1      | Valutazione del Modello di Classificazione . . . . .        | 8         |
| 6.2      | Valutazione della Ricerca . . . . .                         | 9         |
| 6.3      | Limiti della Ricerca . . . . .                              | 10        |
| <b>7</b> | <b>Conclusioni</b>                                          | <b>11</b> |

# 1 Introduzione

Il progetto consiste nello sviluppo di un sistema di Intelligenza Artificiale in grado di leggere un labirinto disegnato a mano, grazie all'utilizzo di una pipeline **OpenCV**, classificare il contenuto delle celle con una *Rete Neurale Convoluzionale* (CNN) e risolvere il gioco tramite l'algoritmo  $A^*$ .

## 1.1 Descrizione dei Vincoli

**Vincolo 1** L'*Agente* può compiere un solo passo alla volta.

**Vincolo 2** L'*Agente* può muoversi solo tra celle adiacenti.

**Vincolo 3** La *posizione iniziale* dell'agente è rappresentata dalla lettera **S**. Nel labirinto non possono esistere posizioni iniziali multiple.

**Vincolo 4** Le celle contenenti una cifra  $n \in \{1, 2, 3, 4\}$ , sono attraversabili a costo  $n$ .

**Vincolo 5** Le celle contenenti la lettera **X** rappresentano *Muri*.

**Vincolo 6** I *Muri* possono essere abbattuti a costo **5**. Una volta abbattuti possono essere attraversati a costo **1**.

**Vincolo 7** Le celle contenenti la lettera **T** rappresentano dei tesori.

**Vincolo 8** Le possibili azioni dell'Agente sono *Movimento verso l'alto*, *Movimento verso il basso*, *Movimento verso destra*, *Movimento verso sinistra*.

**Vincolo 9** Il labirinto è rappresentato da una matrice di dimensione  $N \times N$ . Ogni cella della matrice rappresenta una possibile posizione dell'*Agente*

**Vincolo 10** Sono possibili 2 modalità di gioco. Nella prima modalità, lo scopo è quello di raccogliere tutti i tesori presenti nel labirinto. Nella seconda modalità, lo scopo è quello di raccogliere  $k$  tesori ( $k$  specificato dall'utente). In entrambe le versioni, si cerca il percorso a costo minore.

**Vincolo 11** La matrice del labirinto, così come i contenuti delle singole celle, sono estrapolati in automatico a partire da un'immagine del labirinto stesso.

**Vincolo 12** Uno *Stato* è rappresentato da una *Tupla* di *Posizione attuale*, *Tesori raccolti* e *Muri abbattuti*.

**Vincolo 13** Il costo per attraversare una cella *Tesoro* (T) o *Posizione Iniziale* (S) è **1**.

**Vincolo 14** L'*Agente* non può compiere azioni che lo portino fuori dalla matrice  $N \times N$ .

**Vincolo 15** Lo *Stato Goal* è soddisfatto se il numero di tesori raccolti è maggiore o uguale al numero di tesori desiderati.

## 1.2 Descrizione del Dominio

Il gioco si sviluppa su un labirinto di dimensione  $N \times N$  dove ogni cella rappresenta una possibile posizione dell'*Agente*.

Lo scopo è quello di raccogliere  $k$  tesori, rappresentati dalla lettera **T**, al minor costo possibile.

L'*Agente* parte dalla posizione iniziale **S**, e può spostarsi solo tra celle adiacenti. In ogni caso, l'*Agente* non può superare i limiti della matrice  $N \times N$  che rappresenta il labirinto. Le possibili celle che può incontrare sono:

- **1-4**: Celle attraversabili al costo descritto.
- **X**: Celle di tipo *Muro*. Possono essere abbattuti a costo **5**. Ogni successivo attraversamento di celle muro abbattute avviene a costo **1**.
- **S**: Cella *Posizione Iniziale*. L'agente inizia a muoversi a partire da questa posizione. Può esistere una sola cella di questo tipo all'interno di un labirinto.
- **T**: Cella *Tesoro*. Un tesoro si considera raccolto quando l'*Agente* attraversa una cella di questo tipo. *Tesoro* può essere raccolto una sola volta. Attraversare una cella *Tesoro* ha costo **1**.

## 2 Rappresentazione degli Stati e Spazio degli Stati

Per lo sviluppo del software di risoluzione del problema, si utilizza una rappresentazione ad oggetti derivata dalla classe *Problem* della libreria **AIMA-Python**.

Uno **Stato** è rappresentato come una *Tupla* composta da:

- **Coordinate Agente**: Tupla  $(x, y)$  della posizione attuale.
- **Tesori Raccolti**: Tupla dei tesori già raccolti, fondamentale per il *Goal Test*. Ogni tesoro raccolto è a sua volta rappresentato come una coppia di coordinate  $(x, y)$
- **Muri Abbattuti**: Tupla dei muri già abbattuti e attraversabili a costo **1**, fondamentale per un corretto calcolo dei costi e delle azioni. Ogni muro abbattuto è rappresentato come una coppia di coordinate  $(x, y)$ .

Per quanto riguarda invece lo **Spazio degli Stati**, la sua *cardinalità* è data dal prodotto cartesiano di:

- Tutte le possibili posizioni ( $N \times N$ )
- Tutte le combinazioni possibili di tesori raccolti ( $2^T$ , dove  $T$  corrisponde al numero di tesori presenti nella mappa)
- Tutte le combinazioni di muri abbattuti ( $2^X$ , dove  $X$  corrisponde al numero di muri presenti nella mappa.)

## 2.1 Metodi della classe Problem

Come detto in precedenza, il software fa uso dei metodi della classe **Problem** della libreria **AIMA-Python**, che sono stati adeguatamente implementati per le finalità del progetto.

`actions(state) -> list`: restituisce una lista delle azioni possibili a partire dallo stato `state`. La suddetta lista è rappresentata come un *array* di coordinate che rappresentano lo spostamento (es.  $(-1, 0)$  rappresenta uno spostamento verso l'alto). Le uniche azioni proibite sono quelle che portano l'agente fuori dai confini della mappa.

`result(state, action) -> State`: Dato uno stato `state` e un'azione `action`, restituisce un nuovo stato come risultato dell'azione specificata. Gestisce internamente le componenti dello stato (Tesori raccolti e Muri abbattuti), aggiornando queste componenti qualora la nuova posizione contenga un *Tesoro* non ancora raccolto o un *Muro* non ancora abbattuto.

`path_cost(c, state1, action, state2) -> int`: calcola e ritorna il costo per passare da `state1` a `state2` effettuando l'azione `action`. Il parametro `c` della funzione rappresenta il costo accumulato fino a quel momento.

## 2.2 Stato iniziale e Goal test

Lo **Stato Iniziale** è derivato direttamente dalla fase di *analisi dell'immagine*, ed è rappresentato come coordinate della *Posizione* della lettera *S* nella mappa, e come due tuple vuote per *Tesori raccolti* e *Muri abbattuti*. Il **Goal Test** invece è implementato con il metodo `goal_test(state) -> bool` e ritorna semplicemente **True** nel caso in cui sono stati trovati almeno  $k$  tesori, **False** altrimenti. ( $k$  equivale al numero di tesori da raccogliere, che può essere il numero totale di tesori presenti o il numero di tesori specificati inizialmente).

# 3 Algoritmi di Ricerca

Per la parte di risoluzione del problema, sono stati implementati 2 tecniche di ricerca distinte, una *Informata* e una *Non Informata*, al fine di evidenziare le differenze nella bontà delle soluzioni.

## 3.1 Ricerca Non Informata: Breadth-First Search (BFS)

La tecnica di ricerca *Non Informata* per cui si è optato è la **BFS**, un algoritmo potente ma con alcune limitazioni. Nel dettaglio, l'algoritmo espande i nodi livello per livello, come a creare un cerchio sempre più ampio coinvolgendo ad ogni passaggio un livello in più (rispetto al nodo iniziale). La **BFS** garantisce tuttavia **ottimalità** solo se tutte le azioni possibili hanno lo stesso costo. Nel caso di **TreasureMaze** questo non avviene, di conseguenza non genera performance ottimali.

## 3.2 Ricerca Informata: Algoritmo A\*

Per quanto concerne invece la tecnica di ricerca *Informata*, è stato implementato l'algoritmo **A\***, il quale utilizza una funzione di valutazione  $f(n) = g(n) + h(n)$ , dove:

$g(n)$  rappresenta il *costo effettivo* per raggiungere lo stato  $n$  partendo da 'S'.

$h(n)$  rappresenta l'*euristica* che stima il costo per arrivare al goal.

### 3.3 Euristiche

Sono state implementate 2 euristiche, anche in questo caso per verificare le differenze di performance nelle due casistiche.

**Distanza di Manhattan (base)** Equivale alla distanza tra l'agente e il tesoro più vicino, è semplice ma **ammissibile** in quanto non sovrastima mai il costo.

**Distanza di Manhattan dall' $m$ -esimo tesoro più vicino (avanzata)** In questo caso, viene calcolata la distanza di Manhattan rispetto a tutti i tesori (non ancora raccolti) presenti sulla mappa ( $m$  equivale al numero di tesori ancora necessari). Infine restituisce il valore della distanza dall' $m$ -esimo tesoro più vicino all'agente, cioè la distanza minima che l'agente dovrà compiere per raccogliere l'obiettivo di  $k$  tesori. Si tratta di un euristica **Ammissibile** poiché non sovrastima mai il costo reale; infatti, per raccogliere  $m$  tesori, l'agente dovrà arrivare almeno all' $m$ -esimo tesoro più vicino. Questa euristica è inoltre **più informata** rispetto alla semplice *Distanza di Manhattan* che guarda solo il tesoro più vicino, in quanto fornisce all'algoritmo una stima di quanti passi manchino per raggiungere l'obiettivo finale (e non il semplice obiettivo di raggiungere il tesoro più vicino), spingendo la ricerca verso zone della mappa più dense di tesori.

## 4 Computer Vision

L'obiettivo della fase di Computer Vision è convertire l'immagine (disegnata a mano o generata digitalmente) in una **Matrice** compatibile con il risolutore. Sono state utilizzate per questo scopo le librerie **OpenCV** (Open Source Computer Vision Library) e **numpy**.

### 4.1 Pre-elaborazione e Binarizzazione

La prima fase prevede la conversione dell'immagine in **scala di grigi** e l'applicazione di filtri per la **rimozione del rumore**.

**Rimozione del rumore** È stato applicato un filtro `medianBlur` con kernel  $5 \times 5$

**Threshold Adattivo** Per contrastare ombre non uniformi (tipiche di immagini scattate in condizioni di luce non ottimali) è stata implementata una tecnica di binarizzazione adattiva con metodo *Gaussiano* (`cv2.ADAPTIVE_THRESH_GAUSSIAN_C`). Il parametro `blockSize` non è statico ma viene calcolato dinamicamente in funzione della risoluzione dell'immagine, per garantire che abbia un valore opportuno indipendentemente dalla qualità dell'immagine (in termini di risoluzione).

## 4.2 Analisi morfologica e Perimetrale

Per garantire che la griglia del labirinto sia interpretata come un unico blocco connesso, sono state applicate operazioni di morfologia:

**Dilation** Utilizzata per ispessire i tratti sottili o interrotti.

**Morphological Close** Operazione di **dilatazione** al fine di chiudere eventuali piccoli buchi nei perimetri delle celle (che porterebbero a mancati rilevamenti delle celle stesse).

**Rettangolo di Contenimento** È stato disegnato un perimetro bianco esterno per ovviare all'eventuale mancanza del perimetro esterno (soprattutto per le immagini create digitalmente).

## 4.3 Segmentazione delle Celle

Per individuare ed estrarre le singole celle dall'immagine, è stata utilizzata la funzione `cv2.findContours` con gerarchia `RETR_TREE`. In questo modo il sistema individua tutti i poligoni chiusi, che sono stati successivamente manipolati:

**Filtraggio Area** Vengono innanzitutto scartate le celle con area troppo piccola (rumore) o troppo grande (sfondo o perimetro generale) basandosi sulla proporzione rispetto all'area totale. Nel dettaglio, vengono considerate possibili celle quelle con area compresa tra il **3%** e il **15%** dell'area totale dell'immagine.

**Approssimazione Poligonale** Viene utilizzata la funzione `cv2.approxPolyDP` per identificare i quadrilateri convessi (con 4 vertici).

**Prospettiva** Per ogni cella individuata, viene calcolata la **matrice di trasformazione omografica** con la funzione `cv2.getPerspectiveTransform` per rettificare la prospettiva dell'immagine. Ogni cella viene estratta come immagine quadrata di  $28 \times 28$  pixel.

**Pulizia Margini** A causa della trasformazione delle celle dovuta alla prospettiva, può capitare che parte dei bordi delle celle finiscano all'interno delle celle stesse andando ad alterare il contenuto. Per ovviare a questo problema è stata inserita una pulizia dei margini di ciascuna cella, che va ad **appiattire i 2 pixel** di margine interno delle celle.

## 4.4 Estrazione della Matrice Simbolica

Le immagini delle celle ottenute (grandi  $28 \times 28$  pixel ciascuna) vengono infine passate singolarmente al **Classificatore**. Viene anche effettuato un controllo di integrità generale sulla griglia estratta, in particolare:

- Si verifica che il numero di celle totali sia un **quadrato perfetto** ( $N \times N$ )
- Si verifica la presenza di un unico punto di partenza **S**

- Si verifica la presenza di almeno un tesoro **T**

Qualora uno o più di questi controlli avesse esito negativo, l'esecuzione verrebbe interrotta poiché impossibile.

## 5 Classificazione

Per il riconoscimento del contenuto delle celle (cifre **1, 2, 3, 4** e lettere **S, T, X**) è stata utilizzata una **Rete Neurale Convolutionale (CNN)**. È stato scelto questo approccio in particolare per la sua efficacia del riconoscimento di *pattern spaziali* (permette di riconoscere i caratteri anche se non si trovano esattamente nella stessa posizione dei dati di training).

### 5.1 Dataset e Pre-Processing

Il modello è stato addestrato su una parte ridotta del dataset **eMNIST**, che estende il classico **MNIST** includendo anche le lettere dell'alfabeto. Poiché allo scopo del progetto sono utilizzati solo i caratteri sopra elencati, dal dataset sono stati estratti solo questi caratteri, riducendo così la durata della fase di Training e migliorando al contempo le performance di predizione. Per ottimizzare il processo di predizione, l'output del modello (un vettore di probabilità di 7 classi) viene convertito nei simboli del dominio tramite il seguente mapping:

- **0-3:** Cifre da '1' a '4' (costi di attraversamento).
- **4:** Lettera 'S' (*Posizione iniziale*).
- **5:** Lettera 'T' (*Tesoro*).
- **6:** Lettera 'X' (*Muro*).

Sono inoltre state applicate le seguenti correzioni:

- **Correzione Orientamento:** Si è reso necessario applicare una **rotazione di 90°** in senso antiorario e un **ribaltamento orizzontale** alle immagini EMNIST originali per allinearle all'orientamento standard.
- **Normalizzazione:** I dati sono stati convertiti in **float32** e normalizzati nel range  $[0, 1]$  per favorire la convergenza della discesa del gradiente.

### 5.2 Architettura del Modello

L'architettura implementata per la classificazione delle immagini  $28 \times 28$  si compone di vari livelli ed è stata ottimizzata per evitare **overfitting**:

1. **Input Layer:** Riceve le immagini iniziali in input.
2. **Data Augmentation:** Integrato direttamente nel modello tramite layer di **RandomRotation** (3%) e **RandomZoom** (10%). Questo rende la rete robusta a foto scattate con diverse angolazioni o distanze.

3. **Feature Extraction:** Due blocchi composti da **Conv2D** (32 e 64 filtri) con attivazione **ReLU** e **MaxPooling2D** per estrarre gerarchicamente tratti semplici e forme complesse.
4. **Classificazione:** Un layer **Flatten** seguito da un **Dropout** al 50% (per forzare la rete a non dipendere da singoli neuroni specifici) e un layer **Dense** finale con attivazione **Softmax** per le 7 classi.

## 5.3 Addestramento

L'addestramento è stato configurato per essere efficiente e prevenire la memorizzazione dei dati di training (overfitting).

- **Loss Function:** è stata utilizzata la *Sparse Categorical Crossentropy*, che calcola la distanza matematica tra la distribuzione di probabilità predetta e quella reale, utile nel caso in cui si hanno più di 2 classi (7 in questo caso).
- **Optimizer:** *Adam*, per una gestione adattiva del Learning Rate.
- **Early Stopping:** Implementato monitorando la *Validation Loss*, ossia la *Loss Function* del *Validation Test*, con una pazienza di **2 epoche**. In questo modo il si interrompe l'addestramento quando, per 2 epoche di fila, il modello smette di migliorare su dati mai visti (quelli del *Validation Test*), ripristinando i migliori pesi ottenuti e scongiurando la possibilità di **overfitting** sui dati di training.

## 6 Risultati e Statistiche

In questa sezione vengono analizzate le prestazioni del sistema, confrontando i risultati ottenuti dai diversi moduli.

### 6.1 Valutazione del Modello di Classificazione

La *rete neurale convoluzionale* è stata addestrata per 15 epoche, ma grazie all'implementazione dell'**Early Stopping**, l'addestramento si è automaticamente interrotto all'ottava epoca per prevenire overfitting, avendo già raggiunto la convergenza ottimale.

- **Accuracy:** Il modello ha raggiunto un'accuratezza finale del **99.17%** sui dati del *Validation Set*, cioè su dati mai visti durante il training.
- **Loss:** Il valore di loss sui dati del *Validation Set* si è stabilizzato a **0.0269**.

Per una valutazione più dettagliata delle prestazioni, è stato generato un *Classification Report* (Tabella 1) sui dati di test (su un totale di 3360 immagini).

- **Precision:** Si attesta tra **0.99** e **1** in tutte le classi, indicando la capacità del modello di non classificare come positiva una classe che è in realtà negativa.
- **Recall:** Anche in questo caso, il valore si attesta tra **0.99** e **1** in tutte le classi. Questo significa che il modello è in grado di individuare praticamente tutti i campioni positivi presenti.

- **F-Measure:** Coerentemente con i dati di *Precision* e *Recall*, assume un valore di **0.99** in tutte le classi del problema.

| Classe       | Precision | Recall | F-Measure | Occorrenze |
|--------------|-----------|--------|-----------|------------|
| 1 (Costo 1)  | 1.00      | 0.99   | 0.99      | 483        |
| 2 (Costo 2)  | 0.99      | 1.00   | 0.99      | 492        |
| 3 (Costo 3)  | 1.00      | 0.99   | 0.99      | 461        |
| 4 (Costo 4)  | 0.99      | 0.99   | 0.99      | 472        |
| S (Start)    | 0.99      | 1.00   | 0.99      | 478        |
| T (Treasure) | 1.00      | 0.99   | 0.99      | 478        |
| X (Muro)     | 0.99      | 1.00   | 0.99      | 496        |

Tabella 1: Performance del classificatore CNN.

Per visualizzare meglio le relazioni tra le classi predette e quelle reali, è stata altresì generata la **Confusion Matrix** (Figura 1).

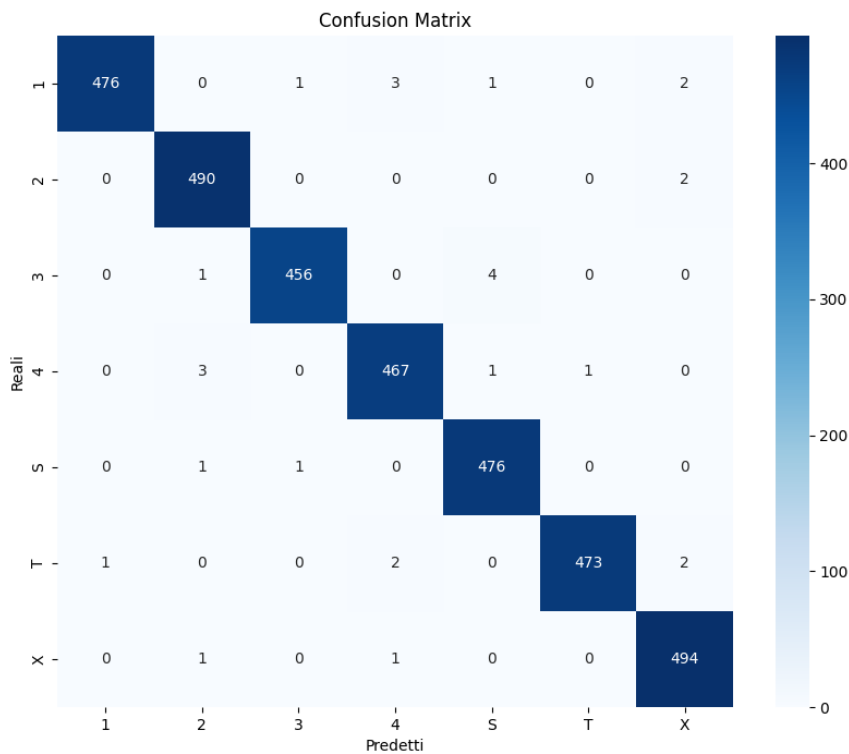


Figura 1: Matrice di Confusione

## 6.2 Valutazione della Ricerca

Per valutare le performance degli algoritmi implementati (BFS, A\* con due euristiche distinte) sono stati effettuati dei test empirici su labirinti di diversa complessità. I parametri

monitorati sono il **tempo di esecuzione**, il **numero dei nodi espansi** e il **costo** del percorso trovato. I dati riportati fanno riferimento ad un labirinto di dimensione  $10 \times 10$  con 5 tesori totali, e  $k$  impostato a **2** (obiettivo trovare il percorso ottimale per raccogliere 2 tesori al costo minore possibile).

Come si evince dai risultati (Tabella 2), la **ricerca non informata** (BFS) espande un numero di nodi notevolmente superiore rispetto ad **A\***. Inoltre, la BFS si limita a trovare il percorso con il minor numero di archi attraversati, ignorando i pesi delle celle. Al contrario, l'**algoritmo A\***, specialmente se guidato dall'euristica avanzata (distanza dall' $m$ -esimo tesoro non ancora raccolto), riduce drasticamente lo spazio di ricerca e garantisce l'individuazione del percorso a costo minimo globale.

| Algoritmo               | Tempo (s) | Nodi Espansi | Costo |
|-------------------------|-----------|--------------|-------|
| BFS                     | 7.9171    | 7048         | 28    |
| A* (Euristica base)     | 0.0415    | 334          | 17    |
| A* (Euristica avanzata) | 0.0094    | 148          | 17    |

Tabella 2: Confronto delle prestazioni empiriche.

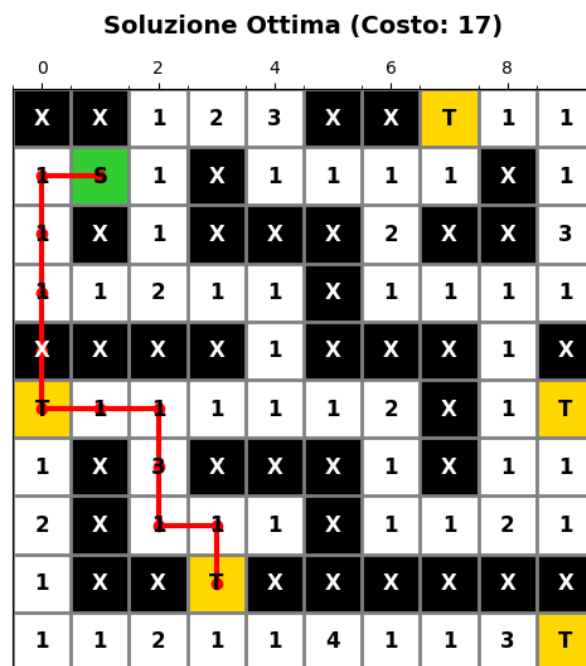


Figura 2: Simulazione

### 6.3 Limiti della Ricerca

Durante i test effettuati su labirinti di dimensioni  $10 \times 10$  con **5 Tesori**, si è osservato un aumento esponenziale del tempo di esecuzione, proporzionale al parametro  $k$  (obiettivo tesori da raccogliere). L'algoritmo **A\*** deve infatti valutare ogni possibile permutazione nell'ordine di raccolta, moltiplicata per le varianti del percorso generate dall'abbattimento dinamico dei muri. Questo genera milioni di nodi e satura la capacità di calcolo.

## 7 Conclusioni

Il progetto ha permesso di affrontare e risolvere un problema eterogeneo, dimostrando l'efficacia dell'integrazione tra tecniche di **Computer Vision**, **Reti Neurali** e **algoritmi di ricerca** nello spazio degli stati.

I filtri applicati alle immagini nella fase di **Computer Vision** si sono rivelati efficaci, riuscendo a convertire immagini scattate anche in condizioni di luce e prospettiche non ottimali in matrici con un alto tasso di successo.

Per quanto riguarda invece il riconoscimento dei caratteri, l'implementazione di una **Rete Neurale Convoluzionale** ha portato ottimi risultati, con un'accuratezza nell'identificazione dei caratteri del dataset eMNIST che ha sfiorato il **99%**.

Infine il confronto tra i risultati dei diversi algoritmi di ricerca ha evidenziato come questi facciano la differenza tra riuscire a risolvere o meno problemi più complessi. Anche la scelta dell'**Euristica** si è rivelata impattante nel numero di *Nodi Espansi* dall'algoritmo **A\***, mostrando ancora una volta l'importanza di una progettazione architetturale puntuale.